PARCIAL C — Programación I

Variante B — SQUID GAME

EJERCICIO 1 — Kiosco de jugadores

Durante una jornada se venden tres productos:

- 1 = Aguita — $1000
- 2 = Sanguche — $2500
- 3 = Galles — $1500

Implementar un menú con las siguientes opciones:

1. Registrar venta
2. Mostrar totales
3. Mostrar producto más vendido
4. Mostrar promedio de unidades por venta
5. Finalizar

Op 1

Pedir:

- producto
- cantidad

La cantidad debe ser mayor que 0

Durante el día no se pueden vender + de **100 unidades** en total. Si una venta supera ese límite, debe rechazarse.

Si es valida:

- acumular unidades por producto
- acumular dinero recaudado
- aumentar en 1 la cantidad de ventas

Op 2

Mostrar:

- cantidad vendida por producto
- total de unidades
- recaudación total

Op 3

Mostrar el producto con mayor cantidad vendida. Si hay empate, informar `HUBO EMPATE`.

Op 4

Calcular:

```
total de unidades / cantidad de ventas
```

Si no hubo ventas, mostrar `SIN DATOS` .

Op 5

Mostrar resumen final y terminar.

Requisitos

- Usar `do...while`
- Usar `switch`
- Validar producto y cantidad

EJERCICIO 2 — El puente de cristal

El jugador debe cruzar un puente de **6 filas**.

En cada fila hay dos paneles:

- Izquierda
- Derecha

Uno es seguro y el otro se rompe. El panel seguro se genera aleatoriamente en cada fila. El jugador comienza con:

- **2 vidas**
- **0 puntos**
- **1 ficha de analisis**

En cada fila

El jugador puede elegir:

1. Izquierda
2. Derecha
3. Usar ficha de analisis

Si elige Izquierda o Derecha

Comparar su eleccion con el panel seguro.

- Si acierta:
  - avanza
  - gana **2 puntos**
- Si falla:
  - pierde 1 vida
  - gana 0 puntos

Si pierde una vida pero todavía le queda al menos una, continúa desde la siguiente fila.

Si usa ficha de analisis

Solo se puede usar si todavía posee la ficha. Al utilizarla:

- el programa informa cual de los dos paneles es seguro
- al elegir correctamente acá gana solo **1 punto**
- si se equivoca pierde 1 vida

Evento especial por fila

Antes de la ronda existe un **20% de probabilidad** de que aparezca la condicion `LUZ APAGADA`

Si ocurre:

- el jugador no puede usar la ficha de análisis en esa fila;
- si elige un panel incorrecto, en vez de perder 1 vida pierde **2 vidas**.

Finalización

El juego termina cuando:

- completa las 6 filas, o
- sus vidas llegan a 0 o menos.

Resultado final

Si completa el puente:

- 10 puntos o más → `SUPERVIVIENTE PERFECTO`
- entre 6 y 9 → `SUPERVIVIENTE`
- menos de 6 → `CRUZO POR POCO`

Si pierde todas las vidas:

```
ELIMINADO
```

Requisitos técnicos

- Usar `rand()` y `srand(time(NULL))`
- Validar las opciones teniendo en cuenta si la ficha está o no disponible
- Resolver correctamente la pérdida doble de vidas durante `LUZ APAGADA`

EJERCICIO 3 — Tarjeta de participante

Cada participante posee un codigo cifrado.

El cifrado original se realizó caracter por caracter.

Letras

Si la letra está en una posicion par de la cadena:

- se desplazó **+3**.

Si está en una posición impar:

- se desplazó **−2**.

La primera posición es la posición 0.

Dígitos

Los dígitos se transformaron así:

- dígito par → se sumó 1
- dígito impar → se restó 1

Ejemplos:

- `2 → 3`
- `7 → 6`

Espacios

Los espacios se reemplazaron por `#`.

Consigna

Desencriptar la cadena. Para las letras:

-posicion par → desplazar **−3**

- posicion impar → desplazar **+2**

Para los dígitos cifrados:

- si el dígito recibido es impar → restar 1
- si el dígito recibido es par → sumar 1

`#` debe volver a convertirse en espacio.

Importante

La posición se cuenta sobre todos los caracteres de la cadena, incluyendo numeros, simbolos y `#`.

Requisitos

- No utilizar `ctype.h`
- Mantener mayúsc y minúsc
- Trabajar con índices

CRITERIOS GENERALES

- Validación de datos.
- Uso correcto del menú en Ejercicio 1.
- Correcta administración de estados, vidas, ficha y eventos en Ejercicio 2.
- Correcto manejo de índices y caracteres en Ejercicio 3.